

Implementation Methodology

| A. Overview of the system topology

1. Analysis of the system components, interactions and the communication protocols used between components

2. Detection of the possible system/components based real-life imperfections that are not explicitly covered in the requirements in order to apply the necessary best practices to handle them later during implementation

a. In this project, handling of the hysteresis due to the possible temperature sensor noise can be given as an example to this step.

| A. Overview of the system topology

3. Definition of the possible fail scenarios and actions to handle each of them

- a. In this project, only one of them is defined and handled. If the EEPROM does not respond properly during initialization, it is reported as fatal error and the sampling does not start at all.

4. Definition of the improvement proposals

- a. In this project, it could be an improvement if a temperature interval between 5 Celcius and 10 Celcius is also defined as a Warning zone.

B. Design and validation of the activity flow

1. Creation of a rough/pseudo activity diagram and tracing it to confirm that the given task can be executed as expected

(In this project the flow was sketched as: power on → init peripherals → read EEPROM once → start timer → loop: ADC → scale → zone → LEDs.)

| B. Design and validation of the activity flow

a. Which operations will take place at each component? In which order? Sequentially or in parallel ?

b. Are there cases where a resource (memory or computational) has to be shared among multiple HW (processors, peripherals etc.) or SW (processes, threads, tasks etc.) sub-components?

c. Validation of the adequacy of the computational budget and the feasibility of meeting the timing requirements

d. Improvements on the activity flow when there is room for optimization to open space (budget) for possible additional features that can come in the future

Budget table (wall time + CPU cycles)

All times must add up to less than 100 microseconds

Assumptions: CPU clock 16 MHz, ADC Kernel clock 4 MHz (/4 prescaler), ADC Sample+Conversion = 41 ADC Cycles (10.25 microseconds)

Step	Clock domain	Wall time (µs)	CPU cycles @ 16 MHz
ISR entry + return	CPU 16 MHz	1.5-2.0	~24-32
Timer clear flag	CPU/bus 16 MHz	~0.15	~10-12
ADC setup	CPU 16 MHz	~0.2	~12-15
ADC wait for EOC	ADC @ 4 MHz, CPU polls @ 16 MHz	~10.25	~164
ADC clear + read DR	CPU 16 MHz	~0.2	~12-15
Temperature Service processing sample	CPU 16 MHz	~0.5-1.0	~40-80
Temperature Service gets zone	CPU 16 MHz	~0.1	~6-10
Led updates via GPIO	CPU/bus 16 MHz	~0.6-1.0	~60-100
Call / alignment overhead	CPU 16 MHz	~0.3-0.5	~20-30
Total (typical)		~13-15 µε	~350-450
Remaining margin		~85-87 µε	~1150-1250

| C. Architectural work

1. Definition of the architectural approach to meet the requirements such as modularity, re-usability, scalability, HW-agnostic design etc.

| a. In this project, a layered design approach with application, service and driver layers is adopted.

2. Definition of the necessary SW components along with the dependencies and interactions among them based on the selected architectural approach

3. Definition of the deployment strategy as optimized as possible

| D. Development work

1. Development of SW components and their interfaces (in case of OOP)

2. Utilization of the necessary mechanisms to meet the timing and memory usage constraints

a. Assignment of the type qualifiers (volatile, const etc.) properly

b. Assignment of the storage class specifiers (static, extern etc.) properly

c. Utilization of the thread synchronization mechanisms (such as mutexes, barriers etc.) or lock-free atomic operations when necessary

| E. Testing

1. Definition of the test cases (preconditions, steps and success criteria) in a way that all requirements will be covered

2. Testing and debugging